Output:

Enter number of vertices : 4

Enter adjacency matrix:

0 1 1 0
1 0 0 1
1 0 0 1
0 1 1 0

Enter starting vertex : 2

BFS Traversal : 2 0 3 1

9b) Write a program to check whether given graph is connected or not using DFS method.

Pseudocode:

```
Start
    Read number of vertices
    Read adjacency matrix
    set visited[i]=0 for i=0 to n-1
    call DFS(0)
Stop
  DFS(v)
     visited[v]=1
     print v
     for i=0 to n-1
        if adj[v][i]==1 and visited[i]==0
           call DFS(i)
  return.
```

code:

```
#include <stdio.h>
#define MAX 10
int visited[MAX];
int adj[MAX][MAX];
int n;
void DFS(int v){
    visited[v]=1;
    printf("%d", v);
    For(int i=0; i<n; i++){
        if(adj[v][i]==1 && !visited[i]){
            DFS(i);
        }
    }
}
```

```
int main() {
    printf("Enter number of vertices:");
    scanf("%d", &n);
    printf("Enter adjacency matrix:\n");
    for(int i=0; i<n; i++) {
        for(int j=0; j<n; j++) {
            scanf("%d", &adj[i][j]);
        }
    }
    for(int i=0; i<n; i++)
        visited[i]=0;
    printf("DFS Traversal starting from vertex 0:\n");
    DFS(0);
    return 0;
}
```

output:

Enter number of vertices: 4

Enter adjacency matrix:

0 1 1 0

1 0 0 1

1 0 0 1

0 1 1 0

DFS Traversal starting from vertex 0:

0 1 3 2

2. Given a file of N employee records with a set k of